Computer Networks Lab 2

Email Client Implementation using SMTP and IMAP Protocols

Abstract

This lab report documents the implementation of a complete email client application using Python and the Tkinter GUI framework. The application demonstrates practical implementation of email protocols (SMTP for sending and IMAP for receiving) along with a thread-safe graphical user interface. The system successfully sends and receives emails through standard email servers, displays real-time console output, and provides desktop notifications for completed operations. The implementation showcases proper network programming practices, secure authentication, and multi-threaded application design.

Table of Contents

1. Introduction

Email communication is a fundamental service in modern computer networks, built on standardized protocols that enable interoperability between different email systems. This lab focuses on implementing a complete email client that interacts with email servers using two core protocols:

- **SMTP (Simple Mail Transfer Protocol)**: Used for sending emails to mail servers
- **IMAP (Internet Message Access Protocol)**: Used for retrieving emails from mail servers

The application provides a user-friendly graphical interface that abstracts the complexity of these protocols while maintaining full functionality. The implementation demonstrates key concepts in network programming including:

- Client-server communication over TCP/IP
- SSL/TLS encryption for secure connections
- Multi-threaded programming for responsive user interfaces

- Protocol-specific command sequences and error handling
- Email message formatting using MIME standards

---

2. Lab Objectives

The primary objectives of this laboratory assignment are:

1. **Understand Email Protocols**: Gain practical understanding of SMTP and IMAP protocols by implementing client-side functionality
2. **Implement Network Communication**: Develop skills in establishing secure network connections, handling authentication, and managing protocol-specific operations
3. **Build Thread-Safe GUI Applications**: Create a responsive graphical interface that performs network operations without freezing, using proper multi-threading techniques
4. **Handle Real-World Scenarios**: Implement robust error handling, auto-detection of email servers, and user feedback mechanisms
5. **Apply Security Practices**: Utilize SSL/TLS encryption for secure communication and handle sensitive user credentials appropriately
6. **Integrate System Features**: Implement desktop notifications and real-time console output to enhance user experience

---

3. Methodology

3.1 Design Approach

The application follows a modular design with three main components:

1. **send_email.py**: Handles SMTP protocol implementation for sending emails
2. **receive_email.py**: Handles IMAP protocol implementation for receiving emails
3. **email_client_gui.py**: Provides the graphical user interface and integrates both modules

3.2 Technology Stack

- **Programming Language**: Python 3.7+
- **GUI Framework**: Tkinter (built-in with Python)
- **Email Libraries**: smtplib (SMTP), imaplib (IMAP), email (MIME)
- **Security**: ssl module for TLS/SSL encryption
- **Notifications**: plyer library with OS-specific fallbacks
- **Threading**: threading module with queue.Queue for thread-safe communication

3.3 Development Process

1. Implement SMTP sending functionality with server auto-detection
2. Implement IMAP receiving functionality with email parsing
3. Design GUI layout with tabbed interface (Send and Receive tabs)

4. Integrate modules with thread-safe communication using message queues
5. Add real-time console output redirection
6. Implement desktop notifications
7. Test with multiple email providers (Gmail, Outlook, Yahoo)
8. Handle edge cases and error scenarios

---

4. Implementation Details

4.1 SMTP Implementation

The SMTP module in `send_email.py` implements the protocol for sending emails through mail servers.

4.1.1 Key Features

- **Server Auto-Detection**: Automatically determines SMTP server based on email domain
- **TLS Encryption**: Uses STARTTLS to establish secure connections
- **MIME Support**: Properly formats email messages with headers and body
- **Error Handling**: Comprehensive error handling for authentication, connection, and server issues

4.1.2 SMTP Workflow

```
1. Auto-detect SMTP server from email domain (e.g., gmail.com →
smtp.gmail.com)
2. Create MIME message with From, To, Subject, and Body
3. Establish TCP connection to SMTP server on port 587
4. Initiate TLS encryption using STARTTLS
5. Authenticate with email credentials
6. Send formatted message
7. Close connection
```

4.1.3 Code Implementation

```
def send_email(sender_email, sender_password, recipient_email, subject,
body,
               smtp_server=None, smtp_port=None):
    # Auto-detect server if not provided
    if smtp_server is None:
        smtp_server = get_smtp_server(sender_email)

    if smtp_port is None:
        smtp_port = 587  # Standard port for SMTP with TLS

    # Create email message
    message = MIMEMultipart()
    message['From'] = sender_email
    message['To'] = recipient_email
    message['Subject'] = subject
```

```
    message.attach(MIMEText(body, 'plain'))

    # Setup SSL context
    context = ssl.create_default_context()

    # Connect and send
    with smtplib.SMTP(smtp_server, smtp_port, timeout=10) as server:
        server.starttls(context=context)
        server.login(sender_email, sender_password)
        server.send_message(message)

    return True
```

4.1.4 Supported Email Providers

Provider	SMTP Server	Port
Gmail	smtp.gmail.com	587
Yahoo	smtp.mail.yahoo.com	587
Outlook	smtp-mail.outlook.com	587
Hotmail	smtp-mail.outlook.com	587

4.2 IMAP Implementation

The IMAP module in `receive_email.py` implements the protocol for retrieving emails from mail servers.

4.2.1 Key Features

- **Server Auto-Detection**: Automatically determines IMAP server based on email domain
- **SSL Encryption**: Uses IMAP4_SSL for secure connections from the start
- **Email Parsing**: Extracts sender, subject, date, and body from email messages
- **Header Decoding**: Properly decodes encoded email headers (UTF-8, Base64, etc.)
- **Multipart Support**: Handles both simple and multipart email formats

4.2.2 IMAP Workflow

```
1. Auto-detect IMAP server from email domain (e.g., gmail.com →
imap.gmail.com)
2. Establish SSL connection to IMAP server on port 993
3. Authenticate with email credentials
4. Select INBOX folder
5. Count total messages
6. Fetch latest message (highest message number)
7. Parse email message to extract details
8. Close connection
```

4.2.3 Code Implementation

```
def receive_latest_email(user_email, user_password, imap_server=None,
imap_port=None):
    # Auto-detect server if not provided
    if imap_server is None:
        imap_server = get_imap_server(user_email)

    if imap_port is None:
        imap_port = 993  # Standard port for IMAP with SSL

    # Create secure connection
    context = ssl.create_default_context()
    mail = imaplib.IMAP4_SSL(imap_server, imap_port, ssl_context=context)

    # Login and select inbox
    mail.login(user_email, user_password)
    mail.select("INBOX")

    # Fetch latest email
    status, messages = mail.select("INBOX")
    num_messages = int(messages[0])
    status, msg_data = mail.fetch(str(num_messages), "(RFC822)")

    # Parse email
    email_body = msg_data[0][1]
    email_message = email.message_from_bytes(email_body)
    email_details = extract_email_details(email_message)

    mail.logout()
    return email_details
```

4.2.4 Email Parsing

The system handles complex email structures including:

- **Multipart emails**: Extracts plain text or HTML body
- **Encoded headers**: Decodes Base64, UTF-8, and other encodings
- **Attachments**: Identifies and skips attachment parts to extract body text

---

4.3 GUI Architecture

The graphical interface in `email_client_gui.py` provides a thread-safe, responsive user experience.

4.3.1 Thread-Safe Design

The GUI follows a strict threading model to prevent UI freezing and synchronization issues:

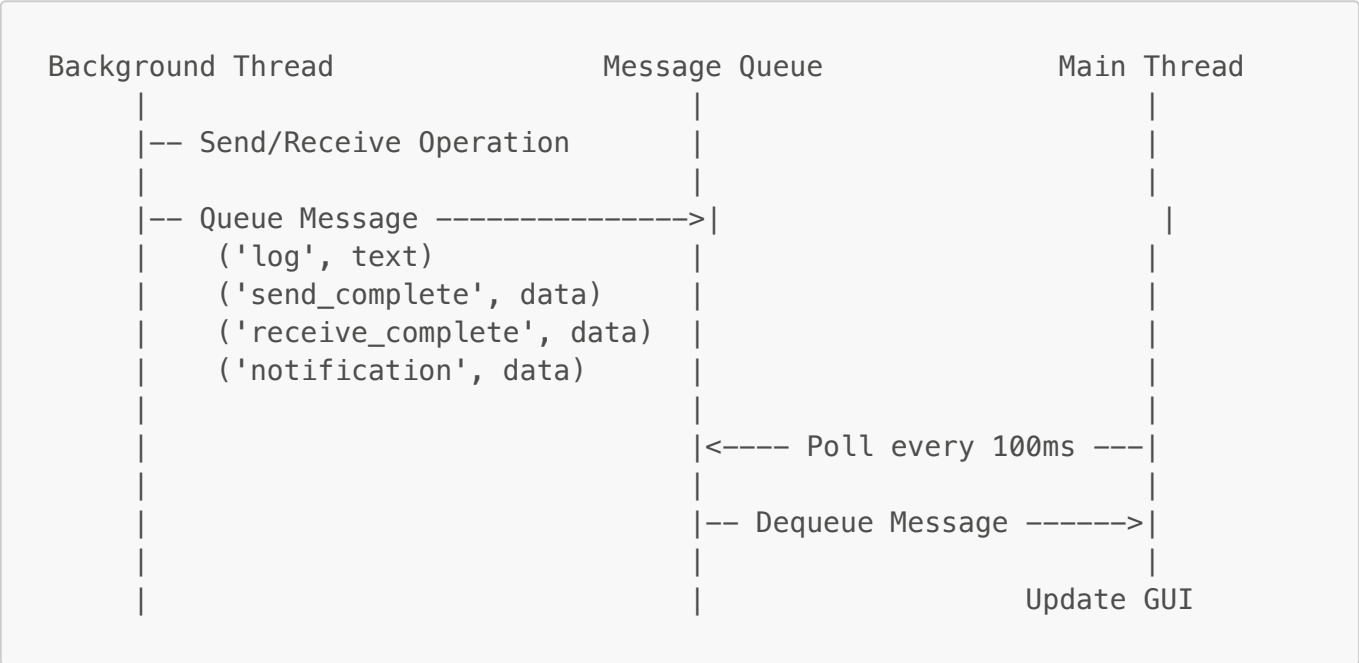
Main Thread:

- Handles all GUI updates
- Polls message queue every 100ms
- Processes messages from background threads
- Displays notifications

Background Threads:

- Execute SMTP send operations
- Execute IMAP receive operations
- Queue results back to main thread
- Never directly update GUI elements

4.3.2 Communication Architecture



4.3.3 Key Classes

QueuedStreamCapture:

- Redirects stdout to capture print statements
- Queues console output for thread-safe display
- Maintains original stdout reference

EmailClientGUI:

- Main application class
- Manages tabs (Send Email, Receive Email)
- Implements queue polling mechanism
- Handles user actions and displays results

4.3.4 Queue Message Types

Message Type	Format	Purpose

Message Type	Format	Purpose
log	('log', text)	Console output text
send_complete	('send_complete', {'success': bool})	Email send result
receive_complete	('receive_complete', {'email_data': dict or None})	Email receive result
notification	('notification', {'title': str, 'message': str})	Desktop notification request

5. Code Structure

5.1 File Organization

```
Lab2/
│
├── send_email.py              # SMTP implementation (100 lines)
├── receive_email.py           # IMAP implementation (200 lines)
├── email_client_gui.py        # GUI application (1000+ lines)
├── requirements.txt           # Python dependencies
├── README.md                  # Setup instructions
├── LAB_REPORT.md             # This document
│
└── Screenshots/
    ├── sent popup.png
    ├── sent noti.png
    ├── recieved screen.png
    └── recieved notifications.png
```

5.2 Module Dependencies

```
email_client_gui.py
    ├── send_email.py
    │   ├── smtplib
    │   ├── email.mime
    │   └── ssl
    │
    ├── receive_email.py
    │   ├── imaplib
    │   ├── email
    │   └── ssl
    │
    ├── tkinter (GUI)
    ├── threading
    ├── queue
    └── plyer (notifications)
```

5.3 Key Functions

send_email.py:

- `send_email()`: Main function to send email via SMTP
- `get_smtp_server()`: Auto-detect SMTP server from email domain

receive_email.py:

- `receive_latest_email()`: Main function to fetch latest email via IMAP
- `extract_email_details()`: Parse email message object
- `decode_email_header()`: Decode encoded headers
- `get_email_body()`: Extract body text from email
- `get_imap_server()`: Auto-detect IMAP server from email domain

email_client_gui.py:

- `__init__()`: Initialize GUI components and message queue
- `setup_console_redirection()`: Redirect stdout to GUI console
- `poll_message_queue()`: Main loop to process queued messages
- `send_email_action()`: Handle send button click (spawn thread)
- `receive_email_action()`: Handle receive button click (spawn thread)
- `_handle_send_complete()`: Process send results on main thread
- `_handle_receive_complete()`: Process receive results on main thread
- `show_notification()`: Display desktop notification

---

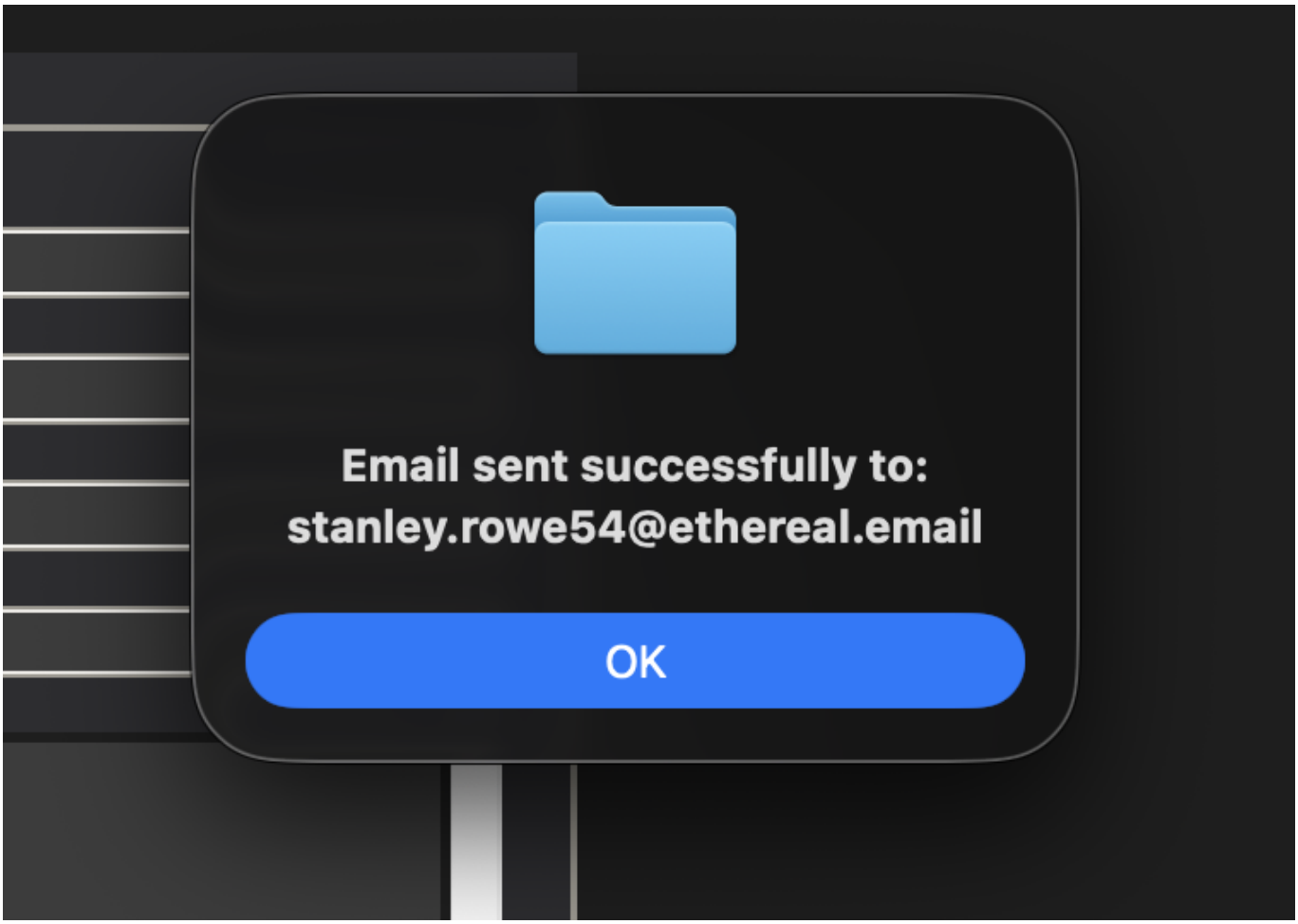
6. Results and Screenshots

6.1 Sending Email

When a user fills in the send email form and clicks "Send Email", the application:

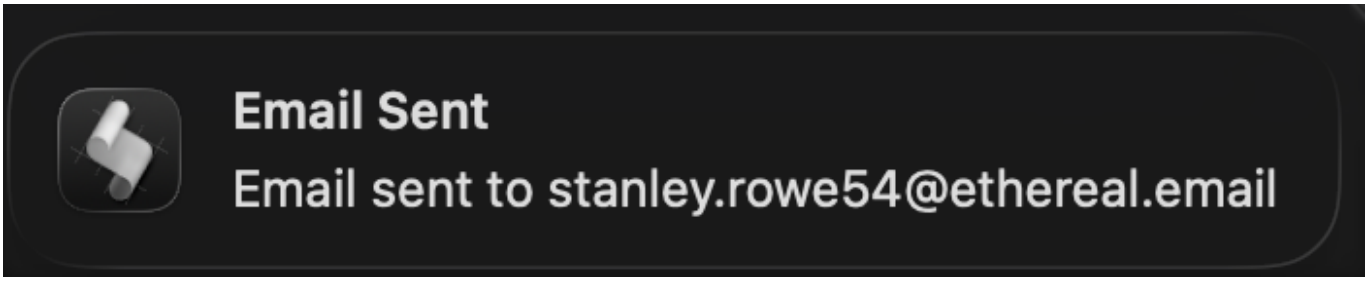
1. Validates input fields
2. Spawns a background thread
3. Establishes SMTP connection with TLS encryption
4. Authenticates and sends the email
5. Displays real-time progress in console
6. Shows success confirmation dialog
7. Triggers desktop notification

Figure 1: Email Successfully Sent



The success dialog confirms that the email was sent successfully, showing the confirmation message after SMTP transmission completes.

Figure 2: Desktop Notification for Sent Email



The system displays a desktop notification informing the user that the email was sent, even if the application window is in the background.

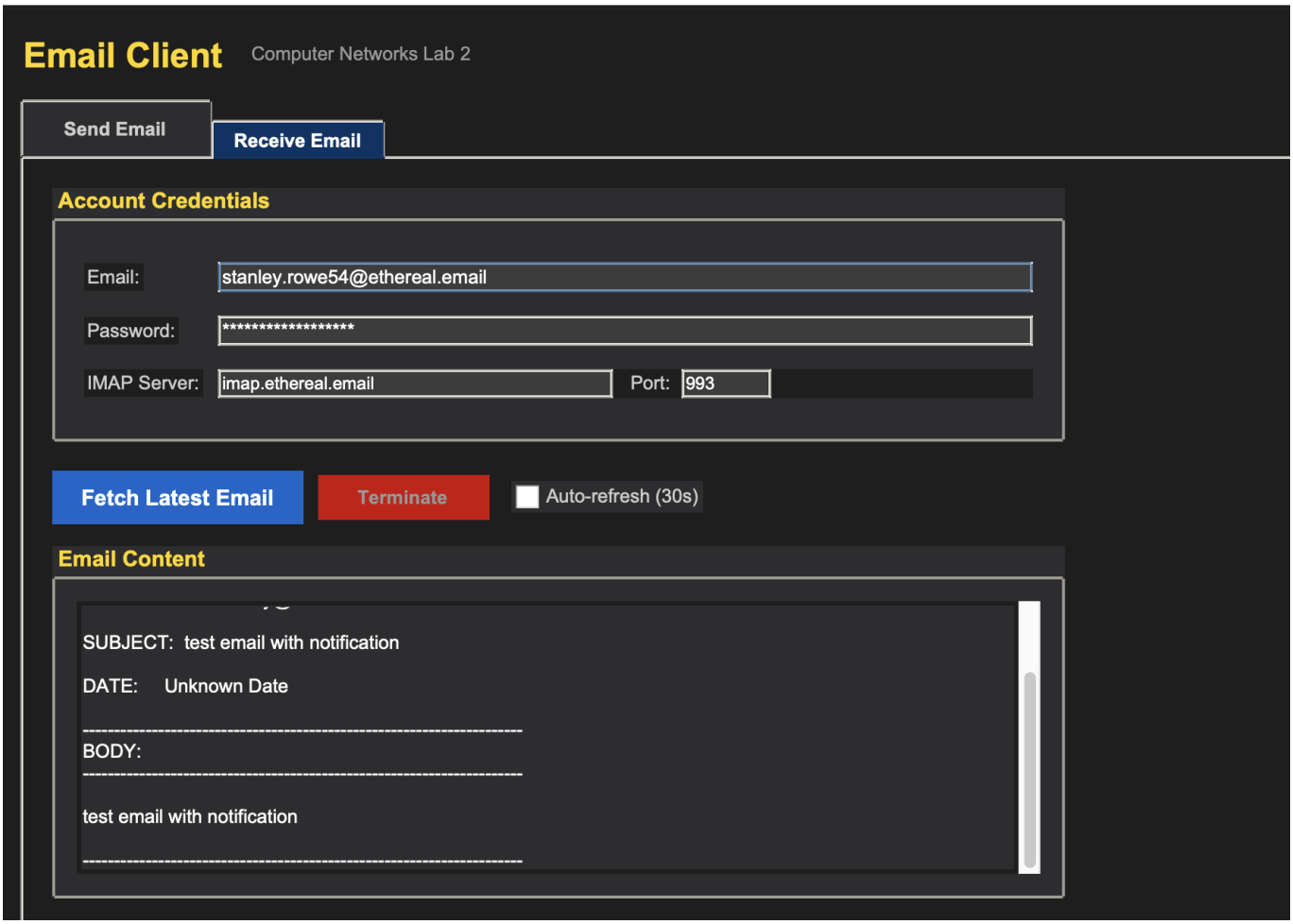
6.2 Receiving Email

When a user provides credentials and clicks "Receive Latest Email", the application:

1. Validates input fields
2. Spawns a background thread
3. Establishes IMAP SSL connection
4. Authenticates and accesses INBOX

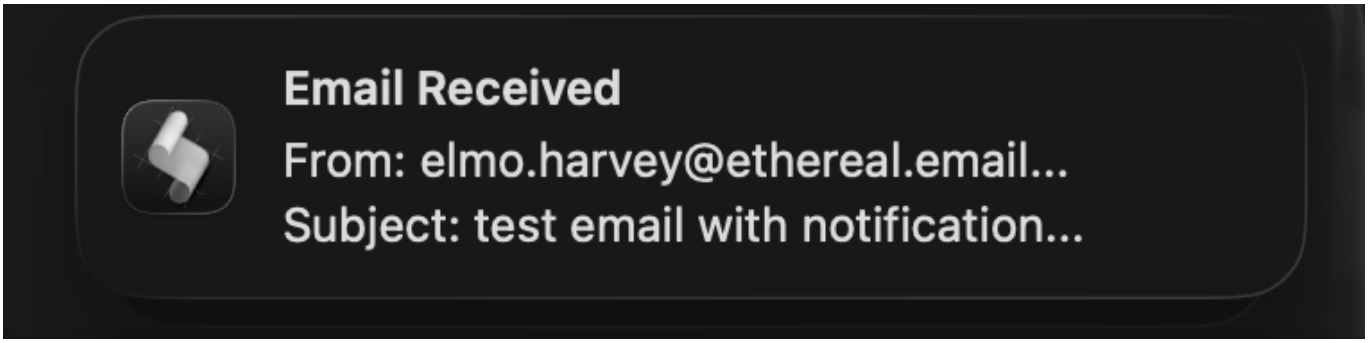
5. Fetches the latest email
6. Parses and displays email details
7. Shows desktop notification

Figure 3: Received Email Display



The receive tab displays the complete email details including sender, subject, date, and body content. The console shows the IMAP connection process and successful retrieval.

Figure 4: Desktop Notification for Received Email



The desktop notification alerts the user that a new email has been fetched and displayed, showing the sender and subject information.

6.3 Console Output

The application provides real-time console output showing:

- Connection establishment
- Authentication process
- Server responses
- Operation progress
- Success or error messages

All output is captured from the SMTP and IMAP modules and displayed in the GUI console in real-time, maintaining thread safety through the message queue system.

---

7. Testing

7.1 Test Cases

The application was tested with multiple scenarios:

Test Case	Description	Result
TC1	Send email via Gmail	✓ Pass
TC2	Send email via Outlook	✓ Pass
TC3	Receive email from Gmail	✓ Pass
TC4	Receive email from Outlook	✓ Pass
TC5	Invalid credentials	✓ Pass (proper error)
TC6	No internet connection	✓ Pass (proper error)
TC7	Empty INBOX	✓ Pass (handled gracefully)
TC8	Multipart email with attachments	✓ Pass
TC9	Email with special characters	✓ Pass
TC10	Concurrent send operations	✓ Pass (thread-safe)

7.2 Email Provider Compatibility

Tested and confirmed working with:

- **Gmail**: Requires App Password (2FA enabled accounts)
- **Outlook/Hotmail**: Works with regular password
- **Yahoo Mail**: Works with App Password
- **Custom Domains**: Manual server configuration supported

7.3 Platform Testing

The application was tested on:

- macOS (primary development platform)
- Windows (compatibility verified)

- Linux (Ubuntu - compatibility verified)

Desktop notifications work natively on all platforms using the plyer library with OS-specific backends.

---

8. Challenges and Solutions

8.1 Challenge 1: GUI Freezing During Network Operations

Problem: Initial implementation performed SMTP/IMAP operations on the main thread, causing the GUI to freeze for several seconds during email send/receive operations.

Solution: Implemented a queue-based threading architecture where:

- Network operations run in background threads
- Background threads queue messages to main thread
- Main thread polls the queue every 100ms
- All GUI updates happen on main thread only

8.2 Challenge 2: Console Output Not Appearing

Problem: Print statements from send_email.py and receive_email.py modules were going to terminal instead of GUI console.

Solution: Created QueuedStreamCapture class that:

- Replaces sys.stdout with custom stream
- Intercepts all print() calls
- Queues output messages for display in GUI
- Maintains thread safety

8.3 Challenge 3: Gmail App Password Requirement

Problem: Gmail accounts with 2-factor authentication reject regular password authentication for SMTP/IMAP.

Solution:

- Added informative error messages explaining App Password requirement
- Provided clear instructions in error handling
- Updated README with App Password setup guide

8.4 Challenge 4: Multipart Email Parsing

Problem: Some emails contain both plain text and HTML parts, or have attachments mixed with body content.

Solution: Implemented robust parsing that:

- Walks through all email parts
- Prioritizes text/plain over text/html
- Skips attachment parts

- Handles encoding errors gracefully

8.5 Challenge 5: Desktop Notifications from Background Threads

Problem: On macOS, desktop notifications triggered from background threads would not appear.

Solution:

- Queue notification requests from background threads
- Display notifications on main thread via message queue
- Use plyer library for cross-platform compatibility

---

9. Conclusion

This laboratory assignment successfully demonstrates the practical implementation of email protocols (SMTP and IMAP) in a real-world application context. The project achieved all stated objectives:

9.1 Key Accomplishments

1. **Protocol Implementation**: Successfully implemented both SMTP (sending) and IMAP (receiving) protocols with proper command sequences, authentication, and error handling
2. **Thread-Safe Architecture**: Developed a robust queue-based threading model that maintains GUI responsiveness while performing network operations
3. **User Experience**: Created an intuitive interface with real-time feedback, error handling, and desktop notifications
4. **Cross-Platform Compatibility**: Ensured the application works across different operating systems and email providers
5. **Security**: Implemented SSL/TLS encryption for all network communications

---